# 10주차_5장

## 5장 토큰화

### 5.0 자연어 처리와 토큰화 개요

- 자연어(Natural Language): 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- 자연어 처리(Natural Language Processing): 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
  - 모호성(ambiguity): 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많음
  - 가변성(Variability):  사투리, 강세, 신조어 작문 스타일로 인해 가변적임
  - 구조(Structure): 구문을 파악하여 의미(semantic)를 해석함
- 문제의 해결: 말뭉치(corpus)를 일정한 단위인 토큰(token)으로 나누어야함
  - 토큰; 개별 단어나 문장 부호와 같은 텍스트, 말뭉치보다 작은 단위
  - 토큰화: 토큰으로 나눈 과정
  - 토크나이저(Tokenizer): 텍스트 문자열을 토큰으로 나누는 알고리즘 소프트웨어
    - 공백 분할: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화
    - 정규표현식 적용: 정규 표현식으로 특정 패턴 식별해 텍스트 분할
    - 어휘 사전: 사전에 정의된 단어 집합을 토큰으로 사용
    - 머신러닝 활용: 데이터세트 기반으로 토큰화하는 방법 학습한 머신러닝 적용
- 어휘 사전: 사전에 정의된 단어를 활용해 토크나이저 구축
  - 직접 어휘 사전 구출 → 없는 단어나 토큰 존재할 수 잇음(OOV - Out of Vocab)
  - 차원의 저주(Curse of Dimensionality): 모든 토큰, 단어 벡터화 → 어휘 사전에 등장하는 토큰 개수만큼 차원 필요 , 벡터값이 거의 모두 0을 가지는 희소 데이터로 표현됨

### 5. 1 단어 및 글자 토큰화

- 토큰화: 텍스트 데이터를 구조적으로 분해해 개별 토큰으로 나누는 작업
- 단어 토큰화: 입력된 텍스트를 단어 단위로 나눔
- 글자 토큰화; 입력된 텍스트를 글자 단위로 나눔

#### 단어 토큰화

- 띄어쓰기, 문장 부호, 대소문자 등 특정 구분자 활용해 토큰화 수행

```
# 5.1 단어 토큰화
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!"
tokenized = review.split()
print(tokenized)
```

- split 메서드 주어진 구분자 통해 문자열 리스트 데이터로 나눠줌
- 구분자 입력하지 않으면 공백(Whitespace) 기준으로 나눔
- 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류에 취약

#### 한국어: 자소 단위 토큰화

- 글자 단위로 문장 학습

- 비교적 작은 단어 사전 구축할 수 있음
- 학습 시 컴퓨터 자원 아낄 수 있음
- 전체 말뭉치 학습할 때 각 단어 더 자주 학습할 수 있음
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용됨

```
# 5.2 글자 토큰화
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)
```

- 공백도 토큰으로 나눔
- 영어 글자 토큰화 → 알파벳으로 나눔
- 한글: 여러 자음과 모음의 조합
- ⇒ 자소 단위로 나누어 자소 단위 토큰화 수행(jamo 라이브러리 활용)

## 5.2 형태소 토큰화

- 형태소 토큰화(Morpheme Tokenization): 언어의 문법 구조를 고려해 텍스트를 형태소 단위로 나눔
- 한국어와 같은 교착어(Agglutinative Language)에서 특히 중요
- 형태소(Morpheme): 실제로 의미를 갖는 최소 단위
  - 자립 형태소(Free Morpheme): 스스로 의미를 가짐. 명사, 동사, 형용사 등 단어의 기본 단위
  - 의존 형태소(Bound Morpheme): 다른 형태소와 결합해야 쓰임; 조사, 어미, 접두사, 접미사

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

```
print("단어:", len(review.split()))
print("글자:", len(list(review)))
print("자소:", len(list(j2hcj(h2j(review)))))
```

### 형태소 어휘 사전과 품사 태깅

- 형태소 어휘 사전(Morpheme Vocabulary): 각 단어의 형태소 정보(+ 품사 정보)를 포함한 사전
- 띄어쓰기 사전: '그는', '나는', '그녀는' 모두 별개 토큰
- 형태소 단위로 만들 경우 '그녀'만 학습하면 '그녀는', '그녀에게' 쉽게 처리 가능
- 품사 태깅(POS Tagging): 각 형태소에 품사를 붙이는 작업
  - 그(명사)+ 는(조사) + 나(명사) + 에게(조사) + 인사()

### KoNLPy

한국어 NLP 라이브러리(pip install konlpy)

- 명사 추출 형태소 분석, 품사 태깅 기능 제공
- 지원 분석기: Okt(open korean text), 꼬꼬마(KKma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab)

| | okt | 꼬꼬마 |
|---|---|---|

| | | |
|---|---|---|
| 명사 추출 | okt.nouns() | kkma.nouns() |
| 구 / 문장 추출 | okt.phrases() | kkma.sentences() |
| 형태소 추출 | okt.morphs() | kkma.morphs() |
| 품사 태깅 | okt.pos() | kkma.pos() |
| 품사 수 | 19개 | 56개 |

Okt 토큰화

```
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)

print("명사 추출 :", nouns)
print("구 추출 :", phrases)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

꼬꼬마 토큰화

```
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("명사 추출 :", nouns)
print("문장 추출 :", sentences)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

### 품사 태그 표 읽기

- N = 체언, 명사류
  - NNG: 보통명사
  - NNP: 고유명사
  - NNB: 의존 명사
  - NP: 대명사
  - NR: 수사
- V = 용언

    - VV: 동사
    - VA: 형용사
    - VX*: 보조용언
    - VCP/VCN: 긍정/부정 지정사
- M = 수식언
    - MD*: 관형사
    - MA*: qntk
- J = 관계언
    - JKS: 주격
    - JKO: 목적격
    - JKM: 부사격
    - JX: 보조사
- E = 어미
    - EF*: 종결
    - EC*: 연결
    - ET*: 전성
    - EP*: 선어말
- X = 접사
    - XPN: 접두사
    - XSN, XSV, XSA: 파생 접미사
    - XR: 어근
- S = 기호
    - SF: 마침표류
    - SP: 쉼표류

### NLTK

- 영어 중심 NLP 라이브러리
- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석

NLTK 패키지 및 모델 다운로드

```
import nltk

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")
nltk.download("punkt_tab")
nltk.download("averaged_perceptron_tagger_eng")
```

- Punkt: 비지도 학습 기반 문장 토크나이저 모델
- Averaged perceptron tagger: 퍼셉트론 기반 품사 태거

```
from nltk import tag
from nltk import tokenize
```

```
sentence = "Those who can imagine anything, can create the impossible."

word_tokens = tokenize.word_tokenize(sentence)
pos = tag.pos_tag(word_tokens)

print(pos)
```

- tokenize.word_tokenize(): 공백 기준 분리
- tokenize.sent_tokenize(): . , !, ? 기준으로 문장 분리
- Tag.pos_tag(): 품사 태깅. 토큰화된 리스트를 넣어야함

### spaCy

- Cython 기반, 빠른 속도와 높은 정확도 목표
- NLTK: 학습 목적, spaCy: 실사용 효율

```
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

형태소 분석의 한계

- 띄어쓰기, 맞춤법이 지켜지지 않은 문장, 신조어, 외래어, 도메인 축약어에 완벽히 대응하지 못함

## 5.3 하위 단어 토큰화

- 필요성: 언어는 계속 변함, 디지털 텍스트에는 오탈자와 띄어쓰기 오류가 많음, 모르는 단어를 적절한 단위로 나누지 못함
- 하위 단어 토큰화(subword tokenization): 단어를 자주 쓰는 하위 단어(subword)의 조합으로 나눔
  - 단어 길이 줄여 처리 속도 향상
  - OOV, 신조어, 은어, 고유어 문제 완화

### 바이트 페어 인코딩(BPE)

다이그램 코딩(Diagram Coding)

- 자주 등장하는 글자 쌍을 새 기호로 치환하는 압축 알고리즘

```
#압축 예시
원문:    abracadabra
Step 1:  AracadAra   ('ab' → A)
Step 2:  ABcadAB     ('ra' → B)
Step 3:  CcadC       ('AB' → C, 치환된 글자도 쌍이 될 수 있음)
```

- 치환 대신 병합된 쌍을 어휘 사전에 추가
- 종료 조건: 더 병합할 쌍이 없거나 정해진 어휘 사전 크기에 도달

**BPE의 학습과 적용**

- 실제 토크나이저는 병합 규칙(merge rules)을 학습된 순서대로 저장
- 새 단어가 들어오면

1. 글자로 쪼갬
2. 학습 때 먼저 배운 병합부터 차례대로 적용

- 센텐스피스(SentencePiece): 구글이 만든 하위 단어 토크나이저 라이브러리/ 유니코드 단위로 처리
- 코프라: 국립국어원, AI hub 등 한국어 말뭉치 쉽게 불러오는 라이브러리

```
# 5.10 데이터 준비
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
dataset = corpus.train
petition = dataset[0]

print("청원 시작일 :", petition.begin)
print("청원 종료일 :", petition.end)
print("청원 동의 수 :", petition.num_agree)
print("청원 범주 :", petition.category)
print("청원 제목 :", petition.title)
print("청원 본문 :", petition.text[:30])
```

```
#5.11
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()

with open("datasets/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

```
#5.12 토크나이저 학습
from sentencepiece import SentencePieceTrainer

SentencePieceTrainer.Train(
    "--input=datasets/corpus.txt\
    --model_prefix=petition_bpe\
    --vocab_size=8000 --model_type=bpe"
)
# 결과: petition_bpe.model (토크나이저), petition_bpe.vocab (어휘 사전)
```

| 매개변수 | 의미 |
| --- | --- |
| `input` | 말뭉치 텍스트 파일 경로 |
| `model_prefix` | 모델 파일 이름 |
| `vocab_size` | 어휘 사전 크기 |
| `character_coverage` | 말뭉치 글자 중 토크나이저가 다룰 글자 비율 |
| `model_type` | `unigram`(기본), `bpe`, `char`, `word` |
| `max_sentence_length` | 최대 문장 길이 |

| 매개변수 | 의미 |
| --- | --- |
| `unk_id` / `bos_id` / `eos_id` | OOV / 문장 시작 / 문장 끝 토큰 ID (기본 0 / 1 / 2) |

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences =
 ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 정수 인코딩 :", encoded_sentence)
print("여러 문장 정수 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(tokenized_sentences)  # [수정] 책은 encoded_sen
tences를 넣었지만 decode_pieces에는 토큰(pieces)을 넣는 것이 맞다
print("정수 인코딩에서 문장 변환 :", decode_ids)
print("하위 단어 토큰에서 문장 변환 :", decode_pieces)
```

| 메서드 | 기능 |
| --- | --- |
| `load(path)` | 모델 불러오기 |
| `encode_as_pieces(문장 or 리스트)` | 하위 단어 토큰으로 분리 |
| `encode_as_ids(...)` | 정수 ID로 인코딩 (어휘 사전 인덱스) |
| `decode_ids(...)` / `decode_pieces(...)` | 정수/토큰 → 원래 문장 |

## 워드피스(WordPiece)

- 빈도가 아닌 확률(점수) 기반으로 병합

$$score(x, y) = \frac{f(x, y)}{f(x)f(y)}$$

- f: 빈도 함수
- x, y: 병합하려는 하위 단어
- f(x, y): 쌍 xy의 빈도

**워드피스 점수의 의미**

$$\frac{P(x, y)}{P(x)P(y)}$$

x와 y가 독립이라면 기대되는 것보다 얼마나 더 자주 붙어나오는지를 잼, 로그 씌우면 PMI

- BPE: 많이 붙어 나오는 쌍 eg. e, s, 처럼 원래 흔한 글자끼리 먼저 붙는 경향
- 워드피스: 서로 강하게 예측하는 쌍, eg. i와 d처럼 거의 항상 함께 나오는 쌍 우선

**실습: 허깅 페이스 토크나이저스**

- 토크나이저스 정규화(normalization)와 사전 토큰화(pre-tokenization) 지원
- 정규화: 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등 텍스트 일관된 형식으로 표준화
- 사전 토큰화: 본 토큰화 전에 공백, 구두점으로 먼저 작게 나눔

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size :", len(vocab))
```

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace

tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()

tokenizer.train(["datasets/corpus.txt"])
tokenizer.save("models/petition_wordpiece.json")
```

정규화 클래스(tokenizers.normalizers)

| 클래스 | 설명 |
|---|---|
| `NFD()`, `NFKD()`, `NFC()`, `NFKC()` | 유니코드 정규화 |
| `Lowercase()` | 소문자 변환 |
| `Strip()` | 양 끝 공백 제거 |
| `StripAccents()` | 악센트 기호 제거 |
| `Replace()` | 문자/정규식 기반 치환 |
| `BertNormalizer()` | BERT 원본 정규화 방식 |
| `Sequence([...])` | 여러 정규화를 순서대로 적용 |

사전 토큰화 클래스(tokenizers.pre_tokenizers)

| 클래스 | 설명 |
|---|---|
| `Whitespace()` | 정규식 `\w+\|[^\w\s]+` 적용 — 단어 경계(공백과 구두점) 기준 |
| `WhitespaceSplit()` | 공백 문자만 기준으로 분리 |
| `Punctuation()` | 구두점 기준 |
| `Digits()` | 숫자 기준 |
| `CharDelimiterSplit()` | 지정 문자 기준 |
| `Metaspace(replacement="_")` | 공백 기준으로 나누고 공백을 `_`로 치환 (센텐스피스 방식) |

| 클래스 | 설명 |
| --- | --- |
| `ByteLevel()` | OpenAI GPT-2 방식. 바이트를 그래픽 문자로 매핑 후 공백 기준 분리 |
| `Split(pattern, behavior)` | 정규식 + 동작( `removed` , `isolated` , `merged_with_previous` , `merged_with_next` ) |
| `Sequence([...])` | 여러 사전 토큰화를 순서대로 적용 |

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder

tokenizer = Tokenizer.from_file("models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()

sentence = "안녕하세요, 토크나이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토크나이저를 사용할 수 있답니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))
```

```
# (추가) NFD로 분해된 토큰을 읽기 쉽게 NFC로 다시 합쳐 보기
import unicodedata

print([unicodedata.normalize("NFC", t) for t in encoded_sentence.tokens])
```